



Separation Assurance in Urban Air Mobility Systems using Shared Scheduling Protocols

Surya Murthy*, Tyler Ingebrand†, Sophia Smith‡, and Ufuk Topcu§
University of Texas at Austin, Austin, Texas, 78712, United States of America

Peng Wei¶
George Washington University, Washington DC, 20052, United States of America

Natasha Neogi||
NASA Langley Research Center, Hampton, Virginia, 23666, United States of America

Ensuring safe separation between aircraft is a critical challenge in air traffic management, particularly in urban air mobility (UAM) environments where high traffic density and low altitudes require precise control. In these environments, conflicts often arise at the intersections of flight corridors, posing significant risks. We propose a tactical separation approach leveraging shared scheduling protocols, originally designed for Ethernet networks and operating systems, to coordinate access to these intersections. Using a decentralized Markov decision process framework, the proposed approach enables aircraft to autonomously adjust their speed and timing as they navigate these critical areas, maintaining safe separation without a central controller. We evaluate the effectiveness of this approach in simulated UAM scenarios, demonstrating its ability to reduce separation violations to zero while acknowledging trade-offs in flight times as traffic density increases. Additionally, we explore the impact of non-compliant aircraft, showing that while shared scheduling protocols can no longer guarantee safe separation, they still provide significant improvements over systems without scheduling protocols.

Nomenclature

Terminology

N	=	Number of agents in the dec-MDP setting.
M	=	dec-MDP tuple.
S	=	Joint state space.
$s \in S$	=	Joint state.
S^i	=	Individual state space for agent i .
$s^i \in S^i$	=	Individual state of agent i .
A	=	Joint action space.
$a \in A$	=	Joint action.
A^i	=	Individual action space for agent i .
$a^i \in A^i$	=	Individual action for agent i .
T	=	Transition function.
R	=	Reward function.
Ω	=	Joint observation space.
Ω^i	=	Individual observation space for agent i .

*Graduate Research Assistant, Oden Institute for Computational Engineering and Sciences, surya.murthy@utexas.edu, AIAA Student Member.

†Graduate Research Assistant, Oden Institute for Computational Engineering and Sciences

‡Graduate Research Assistant, Oden Institute for Computational Engineering and Sciences

§W.A. “Tex” Moncrief, Jr. Chair in Computational Engineering and Sciences VI, Oden Institute for Computational Engineering and Sciences, AIAA Senior Member

¶Associate Professor, Department of Mechanical & Aerospace Engineering, pwei@gwu.edu, AIAA Associate Fellow

||Senior Technologist, Assured Intelligent Flight Systems, natasha.a.neogi@nasa.gov, AIAA Associate Fellow

$\omega^i \in \Omega^i$	= Individual observation for agent i .
O	= Observation function.
π^i	= Individual policy for agent i .
π^{i*}	= Optimal policy for agent i .
$C^i(s^i)$	= Set of critical resource states for agent i at state s^i .
$\beta(s^i, s^j)$	= The blocking function that determines if agent i at state s^i can block agent j from state s^j .
$b(s^i, s^j)$	= The set of pairwise bottleneck states between agent i and agent j .
$B(s^1, \dots, s^N)$	= The set of bottleneck states for all agents given the current joint state of the dec-MDP.
$s^{i'} \in B(s^1, \dots, s^N)$	= Individual bottleneck state for agent i .
γ	= Necessary time for an agent to access a bottleneck.
$P = P^1 \times \dots \times P^N$	= A shared scheduling protocol, where P^i is the decision rule for agent i .
$[t^i, t^i + \gamma]$	= The turn assigned to agent i by the scheduling protocol P .
$X^i(s^i)$	= The set of agents that block agent i at state s^i .

UAM Problem Formulation

x^i	= Position of aircraft i
ϕ	= Flight corridor
U^i	= Route for aircraft i
ϕ_k^i	= k^{th} flight corridor on route U^i for aircraft i
$\lambda(\phi_i, \phi_j)$	= The intersection function that determines if flight corridors ϕ_i and ϕ_j are intersecting.
I	= Intersection of flight corridors
d_{LOS}	= Loss of separation distance

Methodology

ϕ^i	= Current flight corridor of aircraft i
U^i	= Route of aircraft i
v^i	= Velocity of aircraft i
$\dot{v}^i$	= Acceleration of aircraft i
ψ^i	= The heading of aircraft i
h^j	= Information of the neighboring aircraft j state
d_{comm}	= The communication distance between aircraft.
$a_{\text{halt}}, a_{\text{min}}, a_{\text{max}}, a_{\text{maintain}}$	= Speed change actions available to aircraft.
W^i	= Random waiting time used by aircraft i during CSMA/CD.
d_{center}^i	= Distance from aircraft i to the center of an intersection used by SRTF.
ϕ^*	= Priority route defined by Round Robin.

I. Introduction

A critical challenge in air traffic management is ensuring safe separation between aircraft. This challenge intensifies in urban air mobility (UAM) environments, where numerous aircraft operate over urban areas at low altitudes, leading to high traffic density. UAM networks can consist of intersecting flight corridors, which increases the likelihood of conflicts between aircraft at these intersections. UAM environments require a level of precision that exceeds the capabilities of human air traffic controllers. Consequently, incorporating autonomous systems is essential for maintaining safe separation between aircraft.

Existing approaches for UAM separation assurance include strategic separation and tactical separation. Strategic separation methods manage potential conflicts in advance by modifying schedules and routes to balance airspace demand with capacity [1]. However, strategic separation methods lack the flexibility required in dense, dynamic UAM environments. In contrast, tactical separation focuses on real-time deconfliction, enabling aircraft to make immediate speed or path adjustments to avoid conflicts in shared airspace. Recent advancements [2–4] in reinforcement learning enable agents to learn safe navigation policies through repeated simulations, though such methods often require extensive

training and may face challenges in real-world implementation. In this work, we propose a tactical separation approach that leverages shared scheduling protocols to coordinate access to airspace bottlenecks.

We apply shared scheduling protocols to coordinate aircraft movements in UAM environments. Shared scheduling protocols were originally developed for use in early Ethernet networks and operating systems [5] [6], but have recently been adapted for multi-agent reinforcement learning [7]. Shared scheduling protocols were designed to manage access to shared resources and prevent conflicts that arise when multiple agents access a shared resource simultaneously. These shared resources form bottlenecks, where conflicts between agents are likely to occur. Shared scheduling protocols assign turns to different agents, controlling which agents can access bottlenecks at a given time. Shared scheduling protocols prevent conflicts between agents and enhance overall system performance.

UAM networks consist of aircraft navigating along predefined flight corridors to reach their destinations. Bottlenecks naturally form at intersections of these corridors. This increases the risk of separation violations as multiple aircraft attempt to occupy the same airspace simultaneously. We formally identify bottlenecks within the UAM flight network by modeling the system as a decentralized Markov decision process. By observing the bottleneck airspace, shared scheduling protocols issue speed adjustment advisories that coordinate when aircraft pass through intersections. We present three protocols, two of which are fully decentralized, enabling aircraft to maintain safe separation without a central controller.

We evaluate the proposed scheduling protocols in simulated UAM scenarios with constant-speed aircraft, demonstrating their effectiveness in maintaining safe separation and reducing separation violations to zero. We further investigate potential trade-offs, noting that while safety improves, flight times increase with higher traffic density. Finally, we explore the performance of scheduling protocols in settings with non-compliant aircraft, observing a bounded increase in loss of separation events for compliant aircraft as the number of non-compliant aircraft increases.

II. Related Works

A. Safety Assurance

Conflict detection and resolution (CD&R) systems have been a significant focus of research due to their critical role in ensuring safety in dynamic airspace. One notable system is Airborne Collision Avoidance System X (ACAS X), which uses probabilistic modeling to predict future aircraft trajectories and compute optimized avoidance maneuvers in real-time [8]. It offers an improvement over its predecessor, Traffic Collision Avoidance System (TCAS) [9], by incorporating decision logic based on dynamic risk assessment rather than deterministic rule-based responses. Detect and Avoid Alerting Logic for Unmanned Systems (DAIDALUS) [10] detects areas of conflict by predicting aircraft trajectories and provides maneuvering guidance to pilots to prevent or resolve potential conflicts. Reachability analysis approaches compute the set of all possible future states of an aircraft and determine whether any conflicts arise based on these future states [11]. Other CD&R approaches include trajectory optimization approaches [12], dynamic programming [13], and rule-based protocols [14]. These techniques offer provable safety guarantees, making them particularly useful for safety-critical applications such as air traffic management. CD&R approaches operate over relatively short time horizons (2-3 minutes) to resolve individual conflicts between aircraft. In contrast, our work considers deconfliction over longer time horizons through the application of shared scheduling protocols. Rather than resolving immediate conflicts, these protocols aim to prevent loss of separation events by proactively coordinating aircraft access to critical airspace bottlenecks. Importantly, shared scheduling can complement CD&R systems in a layered framework: while scheduling protocols prevent conflicts before they arise, CD&R systems can resolve any remaining conflicts when loss of separation events occur.

Strategic separation management uses pre-planned decisions, including ground delays implemented by air traffic managers. These approaches balance traffic demand with airspace capacity at key bottlenecks such as airport runways, merging points, and air route intersections. Widely used in current air traffic management, strategic separation approaches underpin traffic management initiatives like Ground Delay Programs [15] and Airspace Flow Programs [16]. Demand-capacity balancing techniques use optimization and heuristic methods to schedule flight takeoffs, ensuring that demand matches available capacity at bottlenecks [17]. However, strategic separation approaches often struggle to adapt to real-time variability in aircraft behavior, particularly in dynamic airspace environments. To address this limitation, strategic separation approaches can be integrated with real-time tactical deconfliction methods [1]. Shared scheduling protocols make decisions in real time to prevent conflicts. These protocols have the potential to complement strategic separation approaches by accounting for real-time variance and enhancing operational efficiency.

Unlike strategic separation management, tactical separation approaches aim to resolve conflicts between aircraft in

real time by enforcing safe separation. These approaches include sampling-based methods like Monte Carlo Tree Search [18] and path planning algorithms like A* that identify safe trajectories for multiple aircraft [19]. While these methods focus on resolving conflicts through trajectory planning, we propose shared scheduling protocols as a tactical separation approach that coordinates access to high-conflict areas in UAM environments, hereafter referred to as bottlenecks. By proactively managing entry into these bottlenecks, shared scheduling protocols prevent conflicts and maintain safe operations.

Recently, significant progress has been made in applying deep reinforcement learning methods for tactical separation assurance. Reinforcement learning approaches model the UAM environment as a Markov Decision Process (MDP) and train aircraft to avoid conflict with each other using local observations. Several deep reinforcement learning approaches have been proposed, utilizing long-short term memory networks [2], attention layers [3], and physics-based models [4]. These models have proven effective in resolving conflicts and ensuring safe separation between aircraft in simulated UAM environments. However, deep reinforcement learning methods require an extensive training period during which safety may be compromised. In contrast, shared scheduling protocols do not require training, immediately enforcing safety constraints without training violations.

We consider a UAM flight network similar to Brittain et al. [3], where aircraft travel along pre-specified routes without deviating by changing flight corridors. Aircraft are permitted to adjust their speeds to maintain safe separation from neighboring aircraft. A key distinction between our setting and that of Brittain et al. [3] is that we assume aircraft can halt and hover in midair. This assumption is crucial for applying shared scheduling protocols, as it allows aircraft to wait without occupying bottleneck airspace. Brittain et al. [3] modeled the UAM network as an MDP with each aircraft as an agent trained to use speed adjustments to enforce safe separation constraints via reinforcement learning. While we also model the UAM environment as an MDP, we do not employ reinforcement learning to ensure safe separation. Instead, we assume aircraft follow a baseline policy and rely on shared scheduling protocols to maintain safe separation.

B. Shared Scheduling Protocols

Scheduling protocols are used to coordinate access to shared resources among multiple agents, ensuring that tasks are executed efficiently while minimizing conflicts and maximizing resource utilization. By allocating resources over time, scheduling protocols play a large role in various applications, including operations research [20, 21], and computer operating systems [6, 22]. The primary objective of scheduling protocols is to optimize performance metrics such as task completion time, resource utilization, and fairness among agents, which can vary significantly depending on the specific context and requirements.

Within the field of computer science, scheduling plays a critical role in operating systems and distributed computing environments. Scheduling algorithms such as Round Robin, Shortest Remaining Time First (SRTF), and First-Come, First-Served (FCFS) dictate the order and allocation of CPU time to various processes, influencing system performance and responsiveness [22]. In Round Robin, each process is given a designated time window to access the CPU, allowing for fair time-sharing among processes [6]. The length of time windows in Round Robin must be chosen carefully to ensure that processes have sufficient time to utilize the CPU while ensuring that all processes have the opportunity to access the CPU in a timely fashion [23]. In contrast, SRTF minimizes the average waiting time by prioritizing processes with the least remaining execution time. However, SRTF may lead to longer processes being perpetually denied access to the CPU, which can significantly impact overall system efficiency [23]. FCFS processes tasks in the order they arrive, which can lead to inefficiencies such as the "convoy effect," where short processes wait for a long process to complete, ultimately increasing the average waiting time [22].

Outside of computer science, scheduling protocols are widely applied in operations research to optimize resource allocation and service efficiency. For instance, Round-Robin scheduling has been used in humanitarian logistics to balance resource distribution fairly in disaster response efforts [24]. In public transit systems, such as bus networks, optimization approaches like mixed-integer programming are employed to assign vehicles efficiently to specific routes, improving service reliability and reducing operational costs [20]. In healthcare, scheduling optimization is critical for managing patient admissions, nurse rostering, and operating room allocation [21], with models based on integer programming [25] and metaheuristic algorithms to handle complex and varying demands [26].

More recently, shared scheduling protocols have been applied to multi-agent reinforcement learning (MARL) settings, particularly in decentralized environments where conflicts arise over access to shared resources. Ingebrand et al. [7] define bottlenecks within decentralized Markov decision processes as critical resources that multiple agents need to access but can only be used by a subset at a time. To resolve conflicts at these bottlenecks, the authors propose using decentralized scheduling protocols that allow each agent to independently manage resource access based on local

observations. This approach avoids centralized controllers, which are often impractical in MARL due to scalability issues. By penalizing agents for violating the scheduling protocol, the agents learn to respect resource boundaries and reduce conflicts, improving performance in environments with high agent density. Building on their approach, we extend the use of shared scheduling protocols by modeling the UAM network as a decentralized Markov decision process. We define bottlenecks as airspace intersections where flight corridors converge, creating natural points of conflict that mirror resource contention in discrete MARL environments. We implement scheduling protocols to enable aircraft agents to coordinate access to intersection airspace, with two of the protocols being fully decentralized.

III. Terminology

In this section, we introduce general terms and concepts that underpin our approach, including decentralized Markov decision processes; critical resource states and bottlenecks; and shared scheduling protocols. These concepts provide a flexible framework for modeling and resolving conflicts in a variety of multi-agent systems. While we present these definitions in a general setting here, Section V will adapt them to the Urban Air Mobility (UAM) context. This progression allows us to first establish the foundational concepts before demonstrating how these concepts apply to identifying and managing areas of conflict between aircraft in UAM systems. By structuring the discussion this way, we aim to clearly delineate the general framework from its specific application.

A. Decentralized Markov Decision Process

Multi-agent reinforcement learning (MARL) involves multiple agents interacting within a shared environment, where each agent pursues its own objectives while considering the effects of its actions on both the environment and other agents. Decentralized Markov decision processes (dec-MDPs) model multi-agent environments, where agents make decisions based on their own observations without access to the complete system state [27].

Definition 1 A decentralized Markov Decision Process (dec-MDP) for N agents is a tuple $M = \langle S, A, T, R, \Omega, O \rangle$, where

- 1) $S = (S^1 \times S^2 \times \dots \times S^N)$ is the joint state space, with $s^i \in S^i$ representing the state of agent i ,
- 2) $A = (A^1 \times A^2 \times \dots \times A^N)$ is the joint action space, with $a^i \in A^i$ representing the action of agent i ,
- 3) $T : S \times A \rightarrow S$ is the transition function, mapping the current state $s \in S$ and joint action $a \in A$ to the next state $s' \in S$,
- 4) $R : S \times A \times S \rightarrow \mathbb{R}^N$ is the reward function, with $r^i = R(s, a, s')$ representing the reward received by agent i after transitioning from s to s' upon taking action a ,
- 5) $\Omega = (\Omega^1 \times \Omega^2 \times \dots \times \Omega^N)$ is the joint observation space, with $\omega^i \in \Omega^i$ representing the observation of agent i , and
- 6) $O : S \times A \times S \rightarrow \Omega$ is the observation function, which provides the individual observations $\omega^i \in \Omega^i$ for each agent after taking the joint action a in state s and transitioning to s' .

At a given time t , agents in the dec-MDP exist in a joint state $s_t \in S$. Each agent receives only its own local observation $\omega_t^i \in \Omega^i$ and does not have access to the full joint state. Agents select actions based on their individual observations, and the environment transitions to a new state $s_{t+1} \in S$ based on the joint action $a \in A$. Each agent follows a policy $\pi^i : \Omega^i \rightarrow \mathcal{P}(A^i)$, which maps observations to actions. The goal of each agent is to learn an optimal policy π^{i*} that maximizes its expected future reward. While we assume deterministic transition and observation functions in this work, the framework can be extended to handle stochastic environments.

Definition 2 (Observation Sequence) A joint observation at time t is $\omega_t \in \Omega$. After a joint action $a \in A$ is taken from joint state $s_t \in S$ to next state $s_{t+1} \in S$, the time step increments, resulting in $\omega_{t+1} = O(s, a, s_{t+1})$. Then, $(\omega_0^i, \omega_1^i, \dots, \omega_{\Delta t}^i) \in (\Omega^i \times \dots \times \Omega^i)$ (a Δt number of times) represents a sequence of local decentralized dec-MDP observations of length Δt for agent i . We will represent $(\Omega^i \times \dots \times \Omega^i)$ as $(\Omega^i)^{\Delta t}$ throughout the rest of the paper.

B. Critical Resources and Bottlenecks

In the context of dec-MDPs, critical resources are states that agents must pass through to reach their goal under any optimal policy [7].

Definition 3 (Critical Resource) For an agent i at state $s^i \in S^i$, critical resources $C^i(s^i) \subset S^i$ is the set of states such that $s \in C^i(s^i)$ if under any isolated optimal policy, a rolled-out local trajectory from state s^i that achieves optimal reward will include state s .

A rolled-out local trajectory is the set of states that an agent i will visit under a policy starting at state s^i . Critical resource states are important when considering conflict in MARL systems because agents must pass through the states if they are behaving optimally. Similarly, if an agent is prevented from accessing its critical resource states, then it cannot achieve optimal reward.

We use the definition of critical resources to define bottlenecks in the dec-MDP environment. A bottleneck is a set of critical resource states for an agent that can be used to *block* another agent's critical resources. Blocking occurs when one agent is denied access to critical resource states due to the state of another agent.

Definition 4 (Blocking) The blocking function is defined as $\beta(s^i, s^j) \rightarrow \{True, False\}$. An agent i at state s^i can block agent j from state s^j if $\beta(s^i, s^j) = True$.

The specific blocking function varies between different environments, and we will define blocking in the context of UAM environments in Section IV.

Definition 5 (Pairwise Bottlenecks) Pairwise bottlenecks between a singular agent i in state s^i and agent j ($j \neq i$) in state s^j are defined as $b(s^i, s^j) = \{(s^{i'}, s^{j'}) | s^{i'} \in C(s^i), s^{j'} \in C(s^j), \text{ and } \beta(s^{i'}, s^{j'}) = True\}$.

A pairwise bottleneck occurs when one agent's access to a critical resource prevents another agent from accessing its own critical resource. Since agents must access critical resources to behave optimally, pairwise bottlenecks represent states where conflicts are likely to arise. These pairwise bottlenecks can be combined to form the collection of all bottleneck states.

Definition 6 (Collection of Bottlenecks) For dec-MDP M at time t , agents are at states $s_t^1, s_t^2, \dots, s_t^N$ with critical resources $C^i(s_t^i)$. At time t , the collection of all bottleneck states is $B(s_t^1, \dots, s_t^N) = \bigcup_{i,j} (b(s_t^i, s_t^j)), i \neq j$.

The collection of bottlenecks refers to the set of all pairwise bottlenecks across all agents. From this point onward, when we refer to bottlenecks, we are referring to states within the collection of all bottleneck states otherwise specified.

C. Shared Scheduling Protocols

A scheduling protocol is an algorithm that coordinates access to a bottleneck among multiple agents, ensuring that conflicts are avoided. The protocol assigns the "go" command to a subset of agents, allowing them to enter the bottleneck, while assigning the "wait" command to the remaining agents, preventing their access until their turn. Agents assigned the "wait" command cannot enter bottleneck states until they are explicitly given the "go" command [7].

Shared scheduling protocols use observations over time to determine whether an agent should access a bottleneck. Shared scheduling protocols rely on a necessary time, γ , which represents the minimum duration required for an agent to safely pass through the bottleneck. During this time, the scheduling protocol allocates access to the bottleneck for a single agent, preventing any blocking agents from entering. By sequentially granting access in this manner, the protocol ensures that agents can safely navigate the bottleneck without contention.

Definition 7 (Scheduling Protocol) Given agent i with bottlenecks $\bigcup_j (b(s_t^i, s_t^j)), j \neq i$ and necessary time γ , a scheduling protocol $P = P^1 \times \dots \times P^N$ is defined as a set of decision rules $P^i : (\Omega^i)^t \mapsto \{go, wait\}$. For each agent i , there exists an interval of time $[t^i, t^i + \gamma]$ such that for all $t' \in [t^i, t^i + \gamma]$,

$$P^j(\omega_0^j, \dots, \omega_{t'}^j) = \begin{cases} go & \text{if } j = i \\ wait & \text{if } j \neq i \text{ and } j \in X^i(s^i), \end{cases}$$

where $X^i(s^i)$ is the set of agents that can block agent i .

We note that this definition does not address cases where $j \neq i$ and $j \notin X^i(s^i)$. In such scenarios, the protocol may allow agents to either "go" or "wait" depending on the specific objectives and constraints of the application.

Shared scheduling protocols satisfying Definition 7 guarantee that each agent i is allocated an exclusive time window $[t^i, t^i + \gamma]$ to act within the bottleneck, during which all other agents that could potentially block i are required to wait.

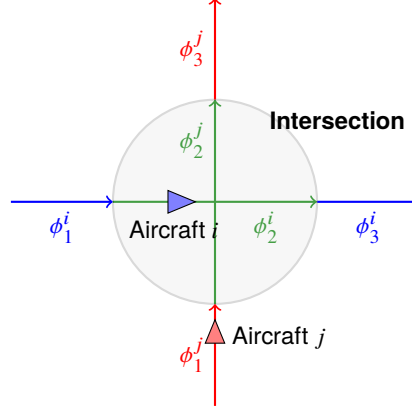


Fig. 1 Illustration of a UAM flight network with intersecting corridors and aircraft positions. The flight corridors $\phi_1^i, \phi_2^i, \phi_3^i$ (blue) and $\phi_1^j, \phi_2^j, \phi_3^j$ (red) represent different routes for aircraft i and j . The intersection, shown as a gray circle with green corridors, occurs where these corridors converge. Aircraft i and j are positioned inside and outside the intersection, respectively.

Ingebrand et al. [7] formalizes the guarantees provided by shared scheduling protocols and proves that any protocol satisfying the given definition resolves conflicts for all agents, provided all agents adhere to the protocol. Since each agent is allocated a dedicated turn to access the bottleneck and no conflicting actions occur during this interval, all agents can sequentially and safely pass through the bottleneck without contention. By ensuring that every agent receives a fair opportunity to act, shared scheduling protocols effectively coordinate bottleneck access across multiple agents.

IV. Problem Formulation

In this section, we formalize the Urban Air Mobility (UAM) environment, outlining the assumptions and objectives that underpin our approach. Specifically, we introduce the concept of UAM flight networks, which consist of predefined flight corridors that aircraft must follow to reach their destinations. Within these networks, regions where flight corridors intersect are identified as critical areas where potential conflicts between aircraft may arise. By defining these elements of the UAM environment, we establish the foundation for mapping the general concepts of decentralized Markov decision processes, critical resource states, and bottlenecks to the UAM setting. These definitions play a central role in modeling aircraft behavior and developing strategies to ensure safe separation at intersections.

A. Urban Air Mobility Flight Network

The Federal Aviation Administration's Concept of Operations for UAM systems introduces UAM flight corridors, designated airspace sections that facilitate point-to-point operations for aircraft [28]. These corridors standardize flight paths, enhancing operational efficiency and reducing societal impacts like noise and congestion.

Definition 8 (Flight Corridor) A flight corridor ϕ is a volume of airspace. An aircraft i at position x^i is in flight corridor ϕ if $x^i \in \phi$.

We assume that flight corridors are *unidirectional*, meaning that aircraft can only fly in one direction through flight corridors. As UAM operations expand, flight corridors can be subdivided and linked to form complex, interconnected flight networks [29]. Within these networks, aircraft follow predetermined routes, defined as sequences of flight corridors, to reach their destinations.

Definition 9 (Routes) A route for aircraft i $U^i = \{\phi_1^i, \phi_2^i, \dots, \phi_m^i\}$ is an ordered set of flight corridors ($\phi_l^i, 1 \leq l \leq m$).

For this paper, aircraft flying along route U^i takes off at the start of ϕ_1 and lands at the end of ϕ_m . In our model, aircraft

cannot switch routes mid-flight by deviating from their intended flight corridors. Intersections are areas where flight corridors intersect, creating opportunities for interactions among aircraft on different routes.

Definition 10 (Intersecting Flight Corridors) *The intersection function $\lambda(\phi_i, \phi_j)$ is defined as:*

$$\lambda(\phi_i, \phi_j) = \begin{cases} \text{True}, & \text{if } \phi^i \cap \phi^j \neq \emptyset, \\ \text{False}, & \text{otherwise.} \end{cases}$$

Flight corridors ϕ_i and ϕ_j are intersecting if $\lambda(\phi_i, \phi_j) = \text{True}$.

By this definition, two flight corridors intersect if their physical airspace overlap (i.e., $\phi^i \cap \phi^j \neq \emptyset$). This means that an aircraft can exist in both flight corridors by flying through the overlapping region. We use the definition of intersecting flight corridors to formally define intersections, which are areas where aircraft on multiple flight corridors can interact.

Definition 11 (Intersection) *An intersection I is a set of flight corridors $\{\phi_1, \phi_2, \dots, \phi_n\}$ satisfying the following properties:*

- 1) $\forall \phi_i \in I, \exists \phi_j \in I, i \neq j$ s.t. $\lambda(\phi_i, \phi_j) = \text{True}$, and
- 2) I is closed under intersection:

$$\phi^* \notin I \implies \lambda(\phi^*, \phi_i) = \text{False}, \forall \phi_i \in I.$$

An aircraft i at position x^i is within the intersection I if it is traveling along one of the flight corridors that comprise the intersection. This means $x^i \in I \iff x^i \in \phi_k^i$ for some $\phi_k^i \in I$.

An intersection I is a set of flight corridors that all intersect with at least one other corridor in I . Any corridor that intersects with a member of I is also included in I . Finally, an aircraft is considered to be in I if it is traveling along any of the corridors in I .

We illustrate this network in Fig. 1, where the flight corridors $\phi_1^i, \phi_2^i, \phi_3^i$ and $\phi_1^j, \phi_2^j, \phi_3^j$ represent the routes for aircraft i and j , respectively. The intersection occurs where these corridors converge and aircraft on different routes interact. While Fig. 1 shows a circular intersection, the shape of the intersection may change depending on the length and position of the intersecting flight corridors. By coordinating when aircraft on different routes pass through intersections, aircraft can maintain a safe separation.

B. Problem Statement

We consider a UAM flight network where aircraft follow predefined routes that intersect at various points. In this model, we assume that aircraft within the same flight corridor can maintain safe separation through speed adjustments. This allows us to focus on enforcing safe separation between aircraft traveling on different corridors. Additionally, we assume that aircraft fly at a constant altitude and rely solely on speed adjustments to maintain separation. This assumption implies that aircraft on the same flight corridor cannot overtake each other, as overtaking would violate the safe separation constraint. Furthermore, we assume that aircraft can wait outside intersection airspace in their corridor, which is crucial for applying shared scheduling protocols. Without this capability, there would be no way to enforce "wait" commands, rendering the protocols ineffective. Our objective is to enforce a minimum safe distance constraint across the network using aircraft speed changes. Specifically, we aim to prevent loss of separation (LOS) events, where aircraft come within a specified distance d_{LOS} of one another.

V. Methodology

In this section, we model the UAM flight network as a decentralized Markov decision process to capture the decision-making dynamics of aircraft navigating the environment. Building on the definitions of flight corridors, routes, and intersections, we formalize the concepts of critical resource states and bottlenecks, which represent key areas of potential conflict under the safe separation objective. By identifying these bottlenecks, we highlight the regions where conflicts between aircraft are most likely to occur. To manage these conflicts, we adapt shared scheduling protocols to the UAM setting, defining rules that coordinate when aircraft can safely traverse bottleneck airspace. This approach integrates the general framework defined in Section III with the specific requirements of UAM environments defined in Section IV, ensuring safe operations.

A. Flight Network as a Decentralized Markov Decision Process

We model the UAM flight network as a decentralized Markov decision process (dec-MDP). We represent each aircraft as an agent and define a joint state space, action space, and observation space. The state space of the UAM dec-MDP is $S = s^1 \times s^2 \times \dots \times s^N$ where the s^i is the local state of each aircraft. We use an augmented version of the state space defined by Brittain et al. [3]. The state of aircraft i is defined as

$$s^i = (x^i, \phi^i, U^i, v^i, \dot{v}^i, \psi^i), \quad (1)$$

where, x^i is the aircraft's position, ϕ^i is the flight corridor, U^i is the route, v^i is velocity, $\dot{v}^i$ is acceleration, and ψ is heading.

Aircraft do not have access to the state information of all other aircraft in the environment. Instead, aircraft observations are comprised of the aircraft's own state information and the state information of neighboring aircraft. The observed state information h^j of neighboring aircraft j is

$$h^j = (x^j, \phi^j, v^j, \dot{v}^j, \psi^j). \quad (2)$$

The joint observation of the UAM dec-MDP is $\omega = \omega^1 \times \omega^2 \times \dots \times \omega^N$. The observation of aircraft i is the state information of aircraft i and the states of neighboring aircraft. Formally the observation of aircraft i is

$$\omega^i = s^i \cup \bigcup_{j: \|x^i - x^j\| \leq d_{\text{comm}}} h^j, \quad (3)$$

where d_{comm} is the maximum communication distance for aircraft. To ensure safety between aircraft, the communication distance must be greater than the LOS distance ($d_{\text{comm}} > d_{\text{LOS}}$). Additionally, we set the communication distance to be greater than the intersection radius. While this isn't required for enforcing safe separation, it provides aircraft with a reasonable view of the intersection before entering.

The action space for each aircraft is comprised of speed change advisories. These include speeding up, slowing down, maintaining current speed, and halting in midair

$$a = (a_{\text{max}}, a_{\text{min}}, a_{\text{maintain}}, a_{\text{halt}}), \quad (4)$$

where

- a_{max} sets the aircraft's airspeed to $v = 60 \text{ kts}$,
- a_{min} sets the aircraft's airspeed to $v = 5 \text{ kts}$,
- a_{maintain} maintains current speed, and
- a_{halt} sets the aircraft's airspeed to $v = 0 \text{ kts}$ and hovers in midair.

We assume that the aircraft's minimum and maximum airspeeds are 5 kts and 60 kts respectively. Speed changes can be used to increase or decrease the separation between aircraft. The a_{halt} action is used by the aircraft to abide by the shared scheduling protocols' "wait" command.

The aircraft's transition function depends on the aircraft's dynamics. Each aircraft has an existing policy that informs the actions it takes as it navigates the environment. We assume that each agent's policy minimizes the time the aircraft spends halting. This is in line with existing work in aircraft energy consumption, which indicates that halting and hovering is the least desirable state for an aircraft due to its large energy consumption [30].

B. Critical Resources and Bottlenecks in the UAM Flight Network

For aircraft traversing the flight network, critical resource states correspond to the future flight corridors for the aircraft's route. Given that aircraft cannot alter their routes mid-flight, all upcoming flight corridors represent states that must be traversed. Formally, for an aircraft i at state s^i traveling along flight corridor ϕ_l^i in route U^i , critical resource states are defined as

$$C^i(s^i) = \{s \mid \phi_k^i \in s \forall k \in \{l+1, m\}\}. \quad (5)$$

This definition means that every subsequent state along a given route is a critical resource state for aircraft i .

To define bottlenecks in the context of the UAM environment, we must first define blocking. In our UAM setting, we associate blocking with the safe separation constraint, which requires a minimum distance between aircraft.

$$\beta(s^i, s^j) = \begin{cases} \text{True} & \text{if } \|x^i - x^j\| < d_{\text{LOS}}, \phi^i \neq \phi^j, v^i \neq 0, \\ \text{False} & \text{else.} \end{cases} \quad (6)$$

According to this definition, aircraft can block critical resource states by flying within the LOS distance of the state with non-zero velocity. Importantly, our definition specifies that blocking only occurs when aircraft are on different flight corridors. It is true that an aircraft can prevent aircraft traveling behind along the same route from reaching shared critical resource states. However, in such cases, the blocking is not reciprocal, since the aircraft traveling behind cannot overtake and prevent the aircraft traveling in front from reaching the critical resource states. We can then define the pairwise bottleneck states for aircraft i and j as

$$b(s^i, s^j) = \{(s^{i'}, s^{j'}) | s^{i'} \in C^i(s^i), s^{j'} \in C^j(s^j), \text{ and } \beta(s^{i'}, s^{j'}) = \text{True}\}. \quad (7)$$

We recognize that this definition of pairwise bottlenecks includes all areas where different flight corridors intersect and are, consequently, subsets of intersection airspace. Therefore, instead of coordinating access to sets of states for each agent, we consider the coordination of access to intersections. An aircraft i is "accessing" an intersection bottleneck I if it is flying through a flight corridor, $\phi^i \in I$, that comprises the intersection at a non-zero airspeed $v^i \neq 0$. We note that in scenarios where no aircraft are traveling along different flight corridors at an intersection, the intersection is no longer a bottleneck. However, aircraft would require global information to make that determination. Since aircraft only have access to local observations, intersections are always considered bottlenecks from the perspective of each aircraft.

We apply shared scheduling protocols to coordinate access to intersection airspace. To improve traffic flow, the UAM scheduling protocols group together aircraft on the same flight corridor into non-conflicting groups, allowing aircraft on the same flight corridor to access the intersection bottleneck at the same time. Aircraft receiving the "go" command from the protocol may pass through intersections freely, while those given the "wait" command must halt using the a_{halt} action and wait until they receive the "go" command.

C. Carrier-Sense Multiple Access with Collision Detection

Carrier-sense multiple access with collision detection (CSMA/CD) is a decentralized scheduling protocol that operates on a first-come first-served basis. If an agent is already accessing a bottleneck, CSMA/CD dictates all conflicting agents should wait. In the UAM setting, this means that if an aircraft is flying through an intersection, all aircraft on different flight corridors should wait. CSMA/CD *collisions* occur when aircraft on adjacent corridors enter the intersection at the same time. These are not the same as actual collisions between aircraft but rather represent *simultaneous access conflicts* that result in delays as aircraft must resolve the contention before proceeding. In such situations, CSMA/CD initializes a random wait time W^i for each aircraft. The random waiting times prevent simultaneous access conflicts from repeatedly occurring, as the first aircraft to stop waiting moves through the intersection while the others must keep waiting [5]. The maximum waiting time parameter must be chosen carefully, since low values increase the probability of repeated conflict, while large values increase the total waiting time. In our simulation, we have aircraft determine their waiting times by uniformly sampling an integer value between 1 and 100 simulation time steps, which correspond to 4 and 400 seconds, respectively. One important property of the UAM environment is that aircraft cannot move backward. When simultaneous access conflicts occur, aircraft must exit the intersection by proceeding along their corridor. Therefore, we decided that aircraft waiting within intersections ($v^j = 0$) are not considered to be accessing the bottleneck by the protocols. Formally, the CSMA/CD protocol for aircraft i is

$$P^i(\omega_0 \dots \omega_t) = \begin{cases} \text{wait} & \text{if } W^i > 0 \\ \text{wait} & \text{if } W^i = 0, \phi^i \notin I, \\ & \text{and } \exists j : \phi^j \in I, \phi^j \notin U^i \text{ and } v^j \neq 0 \\ \text{wait} & \text{if } W^i = 0, \phi^i \in I, \text{ and } v^i = 0 \\ & \text{and } \exists j : \phi^j \in I, \phi^j \notin U^i \text{ and } v^j \neq 0 \\ \text{go} & \text{else.} \end{cases}$$

The CSMA/CD protocol is defined by several cases that determine whether an aircraft i should act or wait based on the current state of the intersection and the activity of other aircraft. First, if the randomized waiting time W^i is greater than zero, the aircraft must wait to reduce the likelihood of repeated simultaneous access conflicts (Case 1). Second, if the aircraft i is not yet in the intersection ($\phi^i \notin I$), it must wait if another aircraft j is moving through the intersection ($v^j \neq 0$) on a different route from i ($\phi^j \notin U^i$) (Case 2). Similarly, if i is already in the intersection ($\phi^i \in I$) but is stationary ($v^i = 0$), it must continue to wait if another aircraft j is moving through the intersection ($v^j \neq 0$) on a different route from i ($\phi^j \notin U^i$) (Case 3). Finally, the protocol allows the aircraft i to proceed through the intersection only if no other aircraft on different routes are moving through the intersection (Case 4).

D. Shortest Remaining Time First

Shortest Remaining Time First (SRTF) is a decentralized protocol like CSMA/CD but uses different logic when simultaneous access conflicts occur. When a conflict occurs, the agent with the shortest remaining time necessary to pass through the bottleneck is allowed to proceed while the other agents must wait [6]. The UAM SRTF implementation handles simultaneous access conflicts by considering the distance of each aircraft to the center of the intersection d_{center} . Aircraft positioned closer to the center of the intersection are assumed to exit more quickly, given a constant velocity profile, corresponding to a shorter necessary time to exit the bottleneck. Formally, the SRTF protocol for aircraft i is

$$P^i(\omega_0^i \dots \omega_t^i) = \begin{cases} \text{wait} & \text{if } \phi^i \in I, \\ & \text{and } \exists j : \phi^j \in I, \phi^j \notin U^i, d_{\text{center}}^j < d_{\text{center}}^i, v^j \neq 0, \\ \text{wait} & \text{if } \phi^i \in I, \\ & \text{and } \exists j : \phi^j \in I, \phi^j \notin U^i, d_{\text{center}}^j = d_{\text{center}}^i, v^j \neq 0, \\ \text{wait} & \text{if } \phi^i \notin I, \\ & \text{and } \exists j : \phi^j \in I, \phi^j \notin U^i \text{ and } v^j \neq 0 \\ \text{go} & \text{else.} \end{cases}$$

The SRTF protocol is defined by several cases that determine whether an aircraft i should act or wait based on its position relative to the intersection center and the activity of other aircraft. First, if i is within the intersection ($\phi^i \in I$), it must wait if there exists another aircraft j also within the intersection ($\phi^j \in I$) on a different route ($\phi^j \notin U^i$), moving ($v^j \neq 0$), and positioned closer to the center of the intersection ($d_{\text{center}}^j < d_{\text{center}}^i$) (Case 1). Case 1 resolves simultaneous access conflicts by prioritizing the aircraft with the shortest remaining time to exit the bottleneck. Secondly, if $d_{\text{center}}^j = d_{\text{center}}^i$, both aircraft will wait (Case 2). In these cases, the protocol can use randomized waiting times, similar to CSMA/CD, to ensure one aircraft proceeds first. Thirdly, if i is not yet in the intersection ($\phi^i \notin I$), it must wait if there exists another aircraft j already within the intersection ($\phi^j \in I$) on a different route ($\phi^j \notin U^i$) and moving ($v^j \neq 0$) (Case 3). This condition mirrors the first-come, first-served logic used in Case 2 of the CSMA/CD protocol. Finally, the protocol allows the aircraft i to proceed through the intersection only if no other aircraft on different routes are moving through the intersection, ensuring a conflict-free passage (Case 4).

E. Round Robin

Round Robin is a turn-based protocol where a subset of agents is allowed to access the bottleneck freely [6]. Unlike CSMA/CD and SRTF, Round Robin is not a decentralized protocol and requires a central authority to keep track of which subset of agents have access to the bottleneck. Round Robin operates by selecting a priority flight corridor $\phi^* \in I$. Aircraft on flight corridor ϕ^* can travel through the intersection freely while aircraft on other corridors must wait. The set of blocking agents is comprised of aircraft on different flight corridors than ϕ^* . Formally, the round-robin protocol is

$$P^i(\omega_0^i \dots \omega_t^i) = \begin{cases} \text{go} & \text{if } \phi^* \in U^i \\ \text{wait} & \text{if } \phi^* \notin U^i. \\ \text{wait} & \text{if } \phi^* \in U^i \text{ and } \exists j : \phi^* \neq \phi^j \text{ and } \phi^j \in I \end{cases}$$

The Round Robin protocol operates by assigning priority to a specific flight corridor ϕ^* , allowing only aircraft on that corridor to pass through the intersection while others must wait. The protocol can be divided into three cases. First, an aircraft i is allowed to proceed if its flight corridor ϕ^i is the priority corridor ϕ^* (Case 1). Second, aircraft i must wait if its flight corridor ϕ^i is not the priority corridor ϕ^* (Case 2). Finally, if there are aircraft on a different flight corridor still in the intersection when the priority switches to ϕ^* , new aircraft attempting to enter on ϕ^* must wait until all aircraft from the previous priority group have exited the intersection (Case 3). This ensures safe coordination when aircraft from the old priority group remain in the bottleneck during a turn switch.

When adapting the Round Robin scheduling protocol to the UAM setting, we assign centralized controllers to each intersection. The aircraft makes entry requests to the controllers when it wants to enter the intersection. The controllers then assign turns to different sets of requesting aircraft based on its route, assigning "wait" and "go" commands depending on which route has the priority. The Round Robin changes priority groups every 100 simulation time steps, which corresponds to 400 seconds.

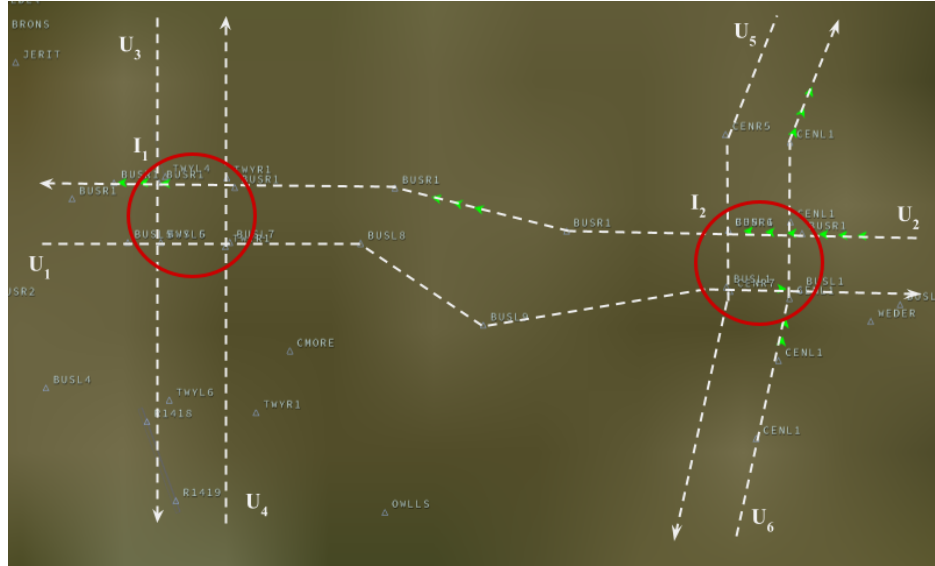


Fig. 2 Simulation setting used for tests, consisting of 6 routes and 2 intersections in the BlueSky air traffic simulator. We evaluate shared scheduling protocols in this scenario, varying traffic densities to assess their performance under different conditions.

VI. Experiments

A. Simulation and Baselines

To test the effectiveness of shared scheduling protocols in urban air mobility (UAM), we use a simulation scenario in the BlueSky Air Traffic Simulator [31]. BlueSky is an open-source air traffic management simulator designed to support research with its accessible, modular design and open-data approach, allowing custom scenario creation through text-based commands. This flexibility allows us to implement and evaluate conflict detection and resolution algorithms without relying on proprietary data or specialized hardware. By leveraging these features, we aim to evaluate the potential benefits and limitations of shared scheduling protocols in managing UAM air traffic *.

We define two hyperparameters for the simulation: a Loss of Separation (LOS) distance of $d_{LOS} = 150$ m and a communication range of $d_{comm} = 1350$ m. The communication distance is equal to the intersection radius, allowing aircraft to detect other aircraft within the intersection before entry, which enables decentralized scheduling protocols to operate effectively. Aircraft follow a default policy of constant speed and altitude, traveling at the maximum allowable airspeed (60 kts) unless modified by a scheduling protocol. To isolate intersection conflicts, we assume that aircraft on the same route can maintain safe separation by adjusting their speed based on the aircraft directly in front of them. Aircraft on different routes take off simultaneously to evaluate shared scheduling protocols in settings without strategic separation.

We examine a UAM network of six routes and two intersections to isolate and showcase the specific effects of each scheduling protocol on intersection safety and traffic flow (Fig. 2). We chose this scenario for its simplicity, allowing us to observe the baseline strengths and limitations of scheduling protocols in scenarios where agents have to pass through multiple bottlenecks. An alternative approach could involve treating pairwise intersections between specific routes as separate intersections. However, we chose to group such intersections together for simplicity and to ensure adequate spacing between consecutive bottlenecks. We test four protocols: no protocol (baseline), Carrier Sense Multiple Access with Collision Detection (CSMA/CD), Round Robin, and Shortest Remaining Time First (SRTF). Each protocol is evaluated across 100 episodes, during which we record three metrics: LOS events, maximum flight times, and halting times. Fewer LOS events indicate reduced risk and lower flight times and halting times indicate higher traffic throughput and efficiency. To further assess scalability, we also vary the number of aircraft on each route to see how the protocols perform under increased traffic densities.

In Fig. 3, we show that the introduction of any shared scheduling protocol reduces the number of LOS events to

*Code for the simulation testing environment is provided at https://github.com/suryakmurthy/UAM_shared_scheduling/

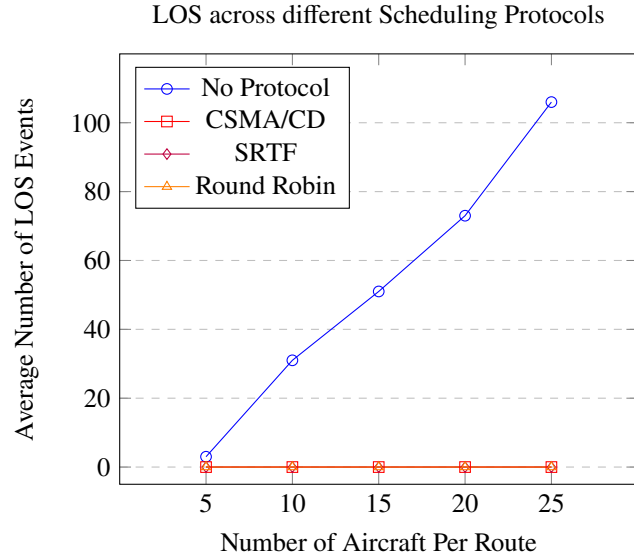


Fig. 3 Average Number of LOS events in Scenarios with increasing numbers of aircraft per route. All shared scheduling protocols reduce LOS events to 0 even as the number of aircraft increases.

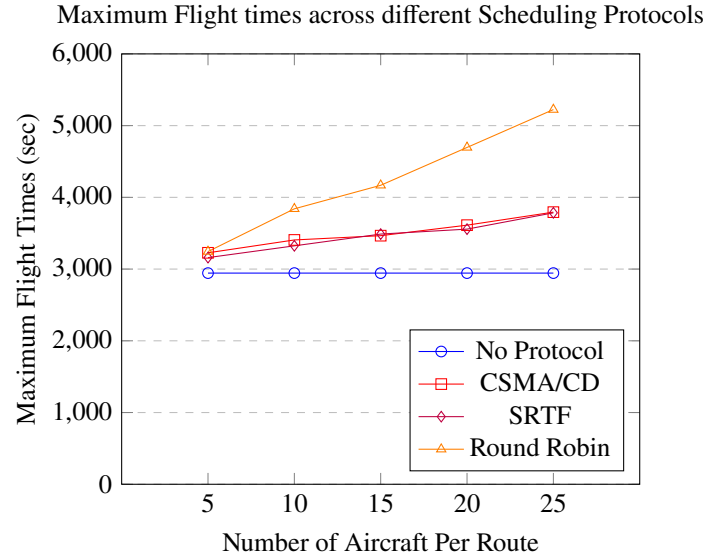


Fig. 4 Average Maximum Flight Times in Scenarios with Increasing Numbers of Aircraft. While protocols can reduce LOS events to 0, we note an increase in aircraft flight times.

Protocol	5 Aircraft	10 Aircraft	15 Aircraft	20 Aircraft	25 Aircraft
CSMA/CD	4.020 %	4.665 %	7.119 %	9.385 %	11.771 %
SRTF	2.393 %	3.669 %	6.230 %	8.372 %	10.820 %
Round Robin	2.785 %	10.653 %	16.170 %	21.936 %	25.983 %

Table 1 Average percentage of flight time spent halting as the number of aircraft per route increases. Halting times increase with traffic density with SRTF having the shortest halting times.

zero. This indicates that the protocols effectively coordinate access to bottlenecks and eliminate conflicts between aircraft. However, in Fig. 4, we observe that these protocols lead to an increase in aircraft flight times. Waiting times at intersections grow as the number of aircraft per route increases, leading to longer travel times. Specifically, CSMA/CD and SRTF protocols lead to a maximum 15% increase in flight times, while Round Robin results in a 60% increase in scenarios with 25 aircraft per route. These results show that shared scheduling protocols can ensure safe separation between aircraft while maintaining a bounded increase in travel times.

B. Results

Further examination of the protocols' impact on flight times is provided in Table 1, where we examine the percentage of time spent halting as the number of aircraft per route increases. Aircraft following CSMA/CD tend to experience larger halting times compared to those following SRTF. This occurs because CSMA/CD resolves simultaneous access conflicts by halting both aircraft, resulting in higher halting times. In contrast, SRTF uses the distance to the center of the intersection as a tiebreaker, resulting in lower halting times. Among all protocols, Round Robin results in the highest halting times. This is due to the protocol's turn-based nature, which can cause aircraft at the same intersection to repeatedly halt, leading to greater worst-case delays. We observe that, in all cases, halting times increase as the number of aircraft per route rises. Higher traffic density at intersections causes longer waiting times, suggesting that integrating traffic flow management strategies, such as demand-capacity balancing, could help reduce halting times for all aircraft.

In scenarios with 5 aircraft, CSMA/CD results in higher average halting times compared to Round Robin due to its reliance on random waiting times. When two aircraft arrive at an intersection simultaneously, CSMA/CD requires each

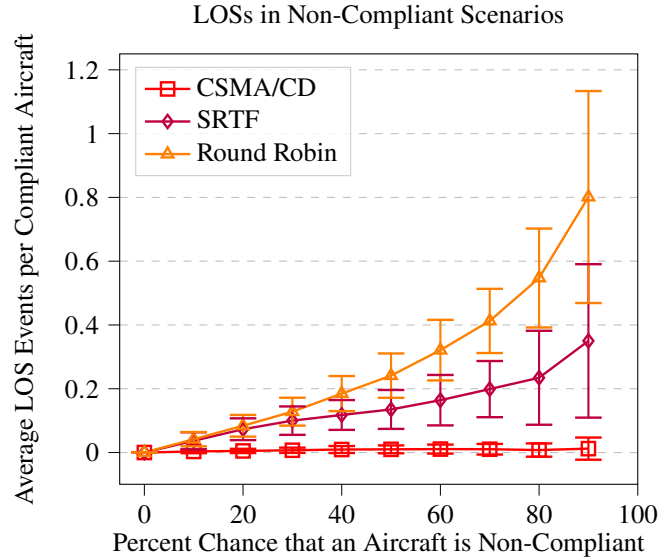


Fig. 5 Number of LOS events at intersections between compliant and non-compliant aircraft. These results are normalized by the number of compliant aircraft. SRTF shows the best performance in settings with high percentages of non-compliant aircraft.

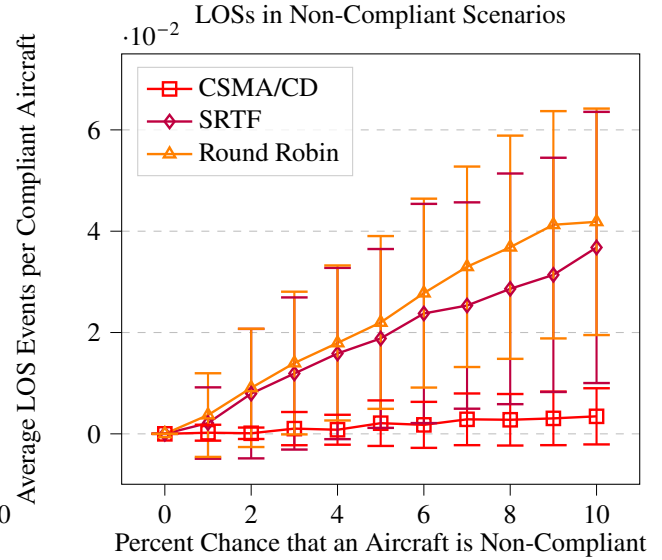


Fig. 6 Number of LOS events at intersections between compliant and non-compliant aircraft. Introducing non-compliant aircraft prevents the scheduling protocols from guaranteeing safe separation between aircraft.

Protocol	10%	20%	30%	40%
CSMA/CD	0.28	0.36	0.45	0.51
SRTF	2.98	5.26	6.3	6.36
Round Robin	3.39	6.04	8.07	9.98
Baseline	51	51	51	51

Table 2 Average number of LOS events between compliant aircraft. Even in scenarios with non-compliant aircraft, the protocols can still achieve a significant improvement over baseline behavior.

to sample a random waiting time between 1 and 100 simulation time steps. Once the waiting time expires, the aircraft may proceed through the intersection if it is clear, leading to halting times that depend on both the duration of the random waiting time and the time it takes for the first aircraft to clear the intersection. In contrast, Round Robin assigns fixed turns of 100 time steps, allowing aircraft from a designated route to pass through the intersection. If no aircraft are waiting on the active route, Round Robin immediately transitions to a route with pending requests, enabling it to complete turns in less than 100 time steps when fewer aircraft are present. However, as traffic density increases, Round Robin requires more turns to allow all aircraft to pass through the intersection, resulting in repeated waiting periods for the same aircraft at the same intersection. Consequently, while Round Robin outperforms CSMA/CD under low traffic conditions, its advantage diminishes as traffic density rises due to increased delays caused by its fixed-turn structure.

C. Non-Compliant Aircraft

In addition to scenarios where all aircraft follow the scheduling protocols, we also explore scenarios involving non-compliant aircraft that follow the flight corridor structure and the constant speed and altitude behavior, but ignore the “wait” commands issued by the scheduling protocol. These aircraft behave as though they have received the “go” command from the protocol at all times, even when they have been asked to “wait”. We simulate a scenario with 15 aircraft per route (90 aircraft total), where each aircraft has a fixed probability of being non-compliant at takeoff, and we measure LOS events between compliant and non-compliant aircraft at intersections, based on data from 100 simulation

episodes. We focus specifically on LOS events between aircraft on different routes at intersections, as these are the conflicts the shared scheduling protocols aim to prevent. In settings without non-compliant aircraft, we assume that aircraft on the same flight corridor can maintain safe separation by observing the aircraft directly in front of them and adjusting their speed to avoid conflicts. However, in non-compliant settings, non-compliant aircraft may overtake compliant aircraft that are halting at intersections, leading to LOS events. Since the protocols were not designed to prevent these LOS events, we exclude conflicts between aircraft on the same route and only consider those between aircraft on different routes. As we demonstrated in the previous results, aircraft that are following the scheduling protocols will not experience LOS events with each other. Therefore, compliant aircraft only experience LOS events with non-compliant aircraft.

In Fig. 6, we observe that even a small number of non-compliant aircraft can compromise the ability of shared scheduling protocols to guarantee safe separation between aircraft. This limitation arises from the assumptions embedded in each protocol: Round Robin assumes that all aircraft will adhere to the assigned turns, while SRTF and CSMA/CD rely on the expectation that aircraft will follow a first-come, first-served order. Consequently, compliant aircraft passing through intersections operate under the assumption that no other aircraft from conflicting routes will enter simultaneously. This fundamental reliance on protocol adherence makes these approaches unable to guarantee safe separation in scenarios with non-compliant aircraft. In Table 2, we observe that these protocols outperform the baseline behavior (51 LOS events).

Among the protocols, CSMA/CD performs the best in all scenarios with non-compliant aircraft (see Fig. 5). This is largely due to its first-come-first-served nature, which prevents protocol-compliant aircraft from entering occupied intersections. This behavior is also shared by SRTF, which outperforms Round Robin for similar reasons. CSMA/CD's performance improves further because it handles simultaneous access conflicts effectively when non-compliant aircraft enter an intersection already occupied by compliant aircraft. As noted in Section V, CSMA/CD resolves such conflicts by having aircraft wait for a random amount of time before attempting to access the bottleneck again. This ensures that compliant aircraft already in the intersection do not cause LOS events when non-compliant aircraft enter. However, CSMA/CD cannot prevent LOS events entirely, as it does not account for scenarios where non-compliant aircraft enter intersections at locations very close to compliant aircraft, resulting in a small number of observed LOS events.

VII. Conclusion and Future Work

In this work, we presented a new approach for enforcing safe separation between aircraft using shared scheduling protocols. We defined bottlenecks in UAM flight networks as areas where flight corridors intersect and applied shared scheduling protocols to coordinate aircraft access to these bottlenecks, ensuring safe separation in real time. Notably, decentralized protocols such as CSMA/CD and SRTF enable safety enforcement using only local aircraft observations. Numerical experiments demonstrate that integrating these protocols effectively enforces safety constraints with a bounded increase in flight times. In scenarios with non-compliant aircraft, the protocols can no longer guarantee zero LOS events, though they still show significant improvement over baseline performance by reducing the frequency of conflicts compared to systems without scheduling protocols.

Future work will expand upon the baseline explored in this study. A key limitation of shared scheduling protocols is the halting required to coordinate access to intersection airspace. In our experiments, we observed that halting times increase with traffic density at intersections. Future work can address this challenge by exploring strategic separation approaches such as demand-capacity balancing, which can reduce traffic density and, consequently, halting times. Furthermore, shared scheduling protocols could be integrated into reinforcement learning models to optimize halting times by enabling aircraft to reduce speed before reaching intersection boundaries. Finally, future research can modify shared scheduling protocols to account for uncertainty in aircraft observations and improve their robustness in dynamic environments.

Acknowledgments

This work was partially supported by the National Aeronautics and Space Administration (NASA) System-Wide Safety (SWS) program under project "In-time Learning-based Aviation Safety Management System," via grant 80NSSC21M0087, and the University Leadership Initiative (ULI) program under project "Autonomous Aerial Cargo Operations at Scale," via grant 80NSSC21M071 to the University of Texas at Austin. The authors are grateful to NASA project technical monitors and partners for their support. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the project sponsor.

References

- [1] Chen, S., Evans, A. D., Brittain, M., and Wei, P., "Integrated conflict management for uam with strategic demand capacity balancing and learning-based tactical deconfliction," *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [2] Deniz, S., Wu, Y., Shi, Y., and Wang, Z., "A reinforcement learning approach to vehicle coordination for structured advanced air mobility," *Green Energy and Intelligent Transportation*, Vol. 3, No. 2, 2024, p. 100157.
- [3] Brittain, M., Yang, X., and Wei, P., "A Deep Multi-Agent Reinforcement Learning Approach to Autonomous Separation Assurance," , 2020.
- [4] Zhao, P., and Liu, Y., "Physics informed deep reinforcement learning for aircraft conflict resolution," *IEEE Transactions on Intelligent Transportation Systems*, Vol. 23, No. 7, 2021, pp. 8288–8301.
- [5] Committee, I., et al., "Ieee standards for local area networks: carrier sense multiple access with collision detection (csma)," *Technical Report ANSI/IEEE Std 802.3-1985*, 1985.
- [6] Parekh, H. B., and Chaudhari, S., "Improved Round Robin CPU scheduling algorithm: Round Robin, Shortest Job First and priority algorithm coupled to increase throughput and decrease waiting time and turnaround time," *2016 International Conference on Global Trends in Signal Processing, Information Computing and Communication (ICGTSPICC)*, IEEE, 2016, pp. 184–187.
- [7] Ingebrand, T., Smith, S., and Topcu, U., "Decentralized Conflict Resolution for Multi-Agent Reinforcement Learning Through Shared Scheduling Protocols," *2023 62nd IEEE Conference on Decision and Control (CDC)*, IEEE, 2023, pp. 7170–7177.
- [8] Manfredi, G., and Jestin, Y., "An introduction to ACAS Xu and the challenges ahead," *2016 IEEE/AIAA 35th Digital Avionics Systems Conference (DASC)*, IEEE, 2016, pp. 1–9.
- [9] Harman, W. H., "TCAS- A system for preventing midair collisions," *The Lincoln Laboratory Journal*, Vol. 2, No. 3, 1989, pp. 437–457.
- [10] Muñoz, C., Narkawicz, A., Hagen, G., Upchurch, J., Dutle, A., Consiglio, M., and Chamberlain, J., "DAIDALUS: detect and avoid alerting logic for unmanned systems," *2015 IEEE/AIAA 34th Digital Avionics Systems Conference (DASC)*, IEEE, 2015, pp. 5A1–1.
- [11] Tomlin, C., Pappas, G. J., and Sastry, S., "Conflict resolution for air traffic management: A study in multiagent hybrid systems," *IEEE Transactions on automatic control*, Vol. 43, No. 4, 1998, pp. 509–521.
- [12] Farley, T., Kupfer, M., and Erzberger, H., "Automated conflict resolution: A simulation evaluation under high demand including merging arrivals," *7th AIAA ATIO Conf, 2nd CEIAT Int'l Conf on Innov and Integr in Aero Sciences, 17th LTA Systems Tech Conf; followed by 2nd TEOS Forum*, 2007, p. 7736.
- [13] Ding, J., and Tomlin, C., "A dynamic programming approach for aircraft conflict detection," *AIAA Guidance, Navigation, and Control Conference*, 2009, p. 5742.
- [14] Hwang, I., Kim, J., and Tomlin, C., "Protocol-based conflict resolution for air traffic control," *Air Traffic Control Quarterly*, Vol. 15, No. 1, 2007, pp. 1–34.
- [15] Odoni, A. R., "The flow management problem in air traffic control," *Flow control of congested networks*, Springer, 1987, pp. 269–288.
- [16] Libby, M., Buckner, J., and Brennan, M., "Operational concept for airspace flow programs (AFP)," *FAA Air Traffic Organization, Systems Operations Services, Tech. Rep.*, 2005.
- [17] Erzberger, H., and Itoh, E., "Design principles and algorithms for air traffic arrival scheduling," *Tech. rep.*, 2014.
- [18] Wu, P., Yang, X., Wei, P., and Chen, J., "Safety assured online guidance with airborne separation for urban air mobility operations in uncertain environments," *IEEE Transactions on Intelligent Transportation Systems*, Vol. 23, No. 10, 2022, pp. 19413–19427.
- [19] Zhao, P., Erzberger, H., and Liu, Y., "Multiple-aircraft-conflict resolution under uncertainties," *Journal of Guidance, Control, and Dynamics*, Vol. 44, No. 11, 2021, pp. 2031–2049.
- [20] Bunte, S., and Kliwer, N., "An overview on vehicle scheduling models," *Public transport*, Vol. 1, No. 4, 2009, pp. 299–317.

- [21] Abdalkareem, Z. A., Amir, A., Al-Betar, M. A., Ekhan, P., and Hammouri, A. I., "Healthcare scheduling in optimization context: a review," *Health and Technology*, Vol. 11, 2021, pp. 445–469.
- [22] Silberschatz, A., and Galvin, P. B., *Operating System Concepts*, John Wiley & Sons, Inc., 2000.
- [23] Tanenbaum, A. S., and Bos, H., *Modern operating systems*, Pearson Education, Inc., 2015.
- [24] Mishra, B. K., Dahal, K., and Pervez, Z., "Dual-mode round-robin greedy search with fair factor algorithm for relief logistics scheduling," *2017 4th International Conference on Information and Communication Technologies for Disaster Management (ICT-DM)*, IEEE, 2017, pp. 1–7.
- [25] Santibáñez, P., Aristizabal, R., Puterman, M. L., Chow, V. S., Huang, W., Kollmannsberger, C., Nordin, T., Runzer, N., and Tyldesley, S., "Operations research methods improve chemotherapy patient appointment scheduling," *The Joint Commission Journal on Quality and Patient Safety*, Vol. 38, No. 12, 2012, pp. 541–AP2.
- [26] Aringhieri, R., Landa, P., Soriano, P., Tanfani, E., and Testi, A., "A two level metaheuristic for the operating room scheduling and assignment problem," *Computers & Operations Research*, Vol. 54, 2015, pp. 21–34.
- [27] Becker, R., Zilberstein, S., and Lesser, V., "Decentralized Markov Decision Processes with Event-Driven Interactions," *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems*, 2004.
- [28] Fontaine, P., "Urban air mobility (UAM) concept of operations," *Federal Aviation Administration*, Vol. 800, 2023.
- [29] Bharadwaj, S., Carr, S., Neogi, N., and Topcu, U., "Decentralized control synthesis for air traffic management in urban air mobility," *IEEE Transactions on Control of Network Systems*, Vol. 8, No. 2, 2021, pp. 598–608.
- [30] Stahl, P., Seren, T., Röbler, C., and Hornung, M., "Development and performance comparison of optimized electric fixed-wing VTOL UAV configurations," *31st Congress of the International Council of the Aeronautical Sciences*, 2018.
- [31] Hoekstra, J., and Ellerbroek, J., "BlueSky ATC Simulator Project: an Open Data and Open Source Approach," 2016.